Machine Learning 2 (APM_50441_EP/DSAIB - 2026-2027). Lab3

The objectives of the lab

This lab illustrates the third lecture (*Trees, Ensemble Methods, SVM and Tabular Deep Learning*). Its goals are:

- to grow and prune a classification tree (CART),
- to understand why averaging trees works (bagging, random forests, extra trees) and to use the out-of-bag estimate,
- to implement AdaBoost and to tune a gradient boosting (learning rate, number of trees, early stopping),
- to use a support vector machine with a kernel,
- to compare all these methods honestly, with the cost-based gain of Lab 1.

The optional *To go further* part, after the core, can be skipped at first.

Use of AI. AI assistants (e.g. the ones built into Colab or VS Code) are allowed, but you must be able to explain every line you use, and answer the *predict before you run* questions before running the code.

Core (about 2 hours)

Exercise 1 (Classification trees)

CART grows a tree greedily: each split is chosen to decrease an impurity (Gini index by default) as much as possible, until the leaves are pure or too small. A fully grown tree overfits; it is then *pruned* along the sequence of subtrees minimizing the penalized criterion error $+\,\alpha \times$ number of leaves (cost-complexity pruning), α being chosen by cross validation.

1. *Predict before you run:* what are the training and test accuracies of a fully grown tree? Fit it, and report its depth and its number of leaves.
2. Compute the cost-complexity pruning path of the full tree (`cost_complexity_pruning_path`). For about 30 values of α along the path, compute the training accuracy and the 5-fold CV accuracy, and plot them against the number of leaves. Select α by cross validation, and plot the top of the pruned tree.

Exercise 2 (Averaging trees: bagging, random forests and extra trees)

A deep tree has a small bias but a large variance. *Bagging* averages B trees grown on bootstrap samples: the variance of the average of B identically distributed predictors of variance σ^2 and pairwise correlation ρ is $\rho\sigma^2 + (1-\rho)\sigma^2/B$. *Random forests* decrease ρ by choosing each split among a random subset of m features (`max_features`); *extra trees* also draw the thresholds at random.

1. *Predict before you run:* how does the variance of the predicted probabilities change from a single deep tree to the bagging of 100 trees? Measure it: train both on 10 random subsamples (80%) of the training set, and compute, for each test loan, the variance of its predicted probability of `Risk` across the subsamples (averaged over the test loans). Compare also their test accuracies.
2. Random forests: for `max_features` in $\{1, 3, 8, 20, \text{all } p\}$ (p = number of one-hot encoded features; all means bagging), compute the out-of-bag (OOB) accuracy of a forest of 300 trees (`oob_score=True`). Which value is best? Compare with the recommendation $m \approx \sqrt{p}$.
3. Grow a random forest ($m = 8$) with `warm_start=True` from 10 to 500 trees, and plot, as a function of the number of trees, its OOB accuracy, its 5-fold CV accuracy (for a few numbers of trees) and its test accuracy. Is the OOB estimate reliable? What does it cost?

4. Compare the impurity-based importances of the forest (`feature_importances_`) with the permutation importances on the test set (`permutation_importance`, 5 repeats). Which features matter? Why can the two rankings differ?

Exercise 3 (Boosting)

Boosting builds a sequence of *weak* learners, each one focusing on the errors of the previous ones. AdaBoost (Lecture 3 and PC3) uses labels $y_i \in \{-1, 1\}$, weights w_i starting from $1/n$, and at each step t:

- fit a weak learner h_t (here a *stump*, a tree of depth 1) with the weights w, and compute its weighted error $\varepsilon_t = \sum_i w_i \mathbf{1}_{h_t(x_i) \neq y_i}$,
- set $\alpha_t = \frac{1}{2} \log \frac{1-\varepsilon_t}{\varepsilon_t}$,
- update $w_i \leftarrow w_i \exp(-\alpha_t y_i h_t(x_i))$ and normalize the weights,

and predicts sign $(\sum_t \alpha_t h_t(x))$.

1. Complete the function `adaboost` below, run it for 200 steps, and plot its training and test accuracies as functions of the number of steps. Compare with `AdaBoostClassifier(DecisionTreeClassifier(max_depth=1), n_estimators=200)` (its `staged_score`). Note: `scikit-learn`'s `estimator_weights_` are equal to $2\alpha_t$ (another normalization of the same algorithm); compare them with your α_t.

   Gradient boosting generalizes boosting to any differentiable loss: each new tree fits the negative gradient of the loss at the current predictions, and is added with a small *learning rate* (shrinkage). The learning rate and the number of trees must be tuned together; *early stopping* stops adding trees when a validation score does not improve anymore.

2. Fit `GradientBoostingClassifier` (trees of depth 3, 600 trees) with learning rates 1, 0.1 and 0.01, and plot the validation log loss as a function of the number of trees (use `staged_predict_proba` on a validation split of the training set). Then fit a `HistGradientBoostingClassifier` with early stopping: how many iterations does it keep?

Exercise 4 (Support vector machines)

An SVM minimizes the hinge loss with a ridge penalty, $\frac{1}{n} \sum_i \max(0, 1 - y_i f(x_i)) + \lambda \|f\|^2$, with $C \propto 1/(n\lambda)$ (PC2). With a kernel $k(x, x') = \exp(-\gamma \|x - x'\|^2)$, f lives in the space of functions $\sum_i \beta_i k(x_i, \cdot)$, and only the *support vectors* (the observations on or inside the margin) have $\beta_i \neq 0$. The features must be standardized.

- Tune an RBF SVM (`SVC`) on the grid $C \in \{0.1, 1, 10, 100\}$, $\gamma \in \{0.001, 0.01, 0.1, 1\}$ with a 3-fold CV of the accuracy. Plot the CV accuracy on the grid, and the number of support vectors as a function of C (for the best γ). Comment.

Exercise 5 (An honest comparison with the cost-based gain)

We finally compare the methods with the criterion that matters for the bank: the real-world gain of the Bayes decision computed from their probabilities (`cost_scorer_from_proba`, Lab 1). All the models are evaluated on the same 5×2 repeated stratified folds, so that their differences can be paired.

- Evaluate the logistic regression, the pruned tree, the random forest, the extra trees, AdaBoost, the histogram gradient boosting and the SVM (its probabilities are obtained by a Platt calibration, `CalibratedClassifierCV(SV ensemble=False)`) on `RepeatedStratifiedKFold(n_splits=5, n_repeats=2)`. Display their mean gains with an interval of ± 2 standard errors, compare the two best ones with the corrected resampled t-test of Lab 1, and evaluate the best one once on the test set.

To go further

The following blocks are optional and independent: each of them only uses the objects defined in the core part. Their difficulty is marked by stars: $\star$ direct extension, $\star\star$ new method, $\star\star\star$ longer or research-flavored.

Block	Lecture	Stars
Impurity measures	CART	$\star$
Gradient boosting from scratch	Gradient boosting	$\star\star$
XGBoost, LightGBM and CatBoost	Boosting implementations	$\star\star$
Stacking	Ensembles	$\star\star$
Random Fourier features	Kernels	$\star\star$
The SVM dual problem	Duality, KKT	$\star\star$
Why do trees beat neural networks on tabular data?	Tabular deep learning	$\star\star$
A tabular foundation model: TabICL	Foundation models	$\star\star$
Synthetic tabular data	Tabular data	$\star\star$
Fairness of the selected model	Metrics and ethics (Lecture 4)	$\star\star$
AutoGluon (separate notebook)	AutoML, stacking	$\star\star$
A tiny prior-fitted network	Foundation models	$\star\star\star$

Exercise 6 (**Impurity measures** $\star$)
A split of a node with proportion p of `Risk` is evaluated by the decrease of an impurity: the misclassification rate $\min(p, 1-p)$, the Gini index $2p(1-p)$ or the entropy $-p\log p - (1-p)\log(1-p)$. The last two are strictly concave, which favors splits creating pure children even when they do not change the majority class.

- Plot the three impurities as functions of p (normalized to have the same maximum). Then compare the CV accuracy of pruned trees grown with the Gini index and with the entropy (`criterion='entropy'`), each with its own α selected by cross validation as in Part 1: the impurity decreases, hence the values of α, do not have the same scale for the two criteria. Plot the CV accuracy against the number of leaves for both criteria.

Exercise 7 (**Gradient boosting from scratch** $\star\star$)
For the logistic loss $\ell(y, F) = \log(1 + e^{-yF})$ with $y \in \{-1, 1\}$, the negative gradient at the current score $F(x_i)$ is $r_i = y_i/(1 + e^{y_i F(x_i)})$. Gradient boosting fits a regression tree to the r_i and adds it with a learning rate ν: $F \leftarrow F + \nu\, h$. (For the squared loss, r_i is the residual: boosting is then *residual fitting*.)

- Implement this gradient boosting with regression trees of depth 3, $\nu = 0.1$ and 200 trees (starting from the log-odds of the training proportion), and compare its test accuracy along the iterations with `GradientBoostingClassifier` (which also updates the values of the leaves with a Newton step).

Exercise 8 (**XGBoost, LightGBM and CatBoost** $\star\star$)
The modern boosting libraries differ by their tree growth (*depth-wise* for XGBoost, *leaf-wise* for LightGBM, *symmetric* trees for CatBoost), their handling of categorical features (CatBoost encodes them with ordered target statistics, LightGBM finds optimal partitions of the categories) and many engineering details. XGBoost uses a second-order expansion of the loss: the optimal value of a leaf is $-G/(H + \lambda)$, where G and H are the sums of the gradients and Hessians of its observations.

- Compare the CV gains (Bayes decision, 5 folds) of XGBoost, LightGBM and CatBoost with their default parameters, the last two with native categorical features (the categorical columns converted to the `category` type), and with the histogram gradient boosting of the core part.

Exercise 9 (**Stacking** $\star\star$)
Instead of choosing one model, *stacking* learns how to combine them: a final model (here a logistic regression) is trained on the out-of-fold predicted probabilities of the base models.

- Stack the logistic regression, the random forest, the gradient boosting and the SVM with `StackingClassifier`, and compare its CV gain with its members.

Exercise 10 (Random Fourier features ⋆⋆)
A kernel SVM manipulates an $n \times n$ kernel matrix. Random Fourier features (Rahimi and Recht) approximate the RBF kernel with D explicit features $z(x) = \sqrt{2/D}\cos(Wx + b)$, $W \sim \mathcal{N}(0, 2\gamma I)$, $b \sim \mathcal{U}[0, 2\pi]$, so that $\mathbb{E}[z(x)^\top z(x')] = \exp(-\gamma\|x - x'\|^2)$: a linear model on $z(x)$ approximates the kernel method at a cost linear in n.

- Compare the test accuracy and the fitting time of the tuned RBF SVM with a logistic regression on $D \in \{50, 200, 1000\}$ random features (`RBFSampler` with the same γ).

Exercise 11 (The SVM dual problem ⋆⋆)
The dual of the soft-margin SVM is $\max_\alpha \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j k(x_i, x_j)$ subject to $0 \leq \alpha_i \leq C$ and $\sum_i \alpha_i y_i = 0$. The KKT conditions show that $\alpha_i = 0$ outside the margin, $0 < \alpha_i < C$ on the margin and $\alpha_i = C$ inside it: the support vectors are the observations with $\alpha_i > 0$.

- On 300 training loans, solve the dual with `scipy.optimize.minimize` (method `SLSQP`) for the RBF kernel and the tuned parameters, and compare your α with `SVC`'s `dual_coef_` (which stores $\alpha_i y_i$ for the support vectors).

Exercise 12 (Why do trees beat neural networks on tabular data? ⋆⋆)
Grinsztajn et al. (2022) identified three inductive biases that favor tree ensembles on tabular data: the target is often irregular (trees can represent sharp steps, neural networks are smooth), many features are uninformative (trees ignore them, neural networks do not), and the features have a meaning on their own (trees are *not* rotation invariant, which helps; neural networks are).

- Train a multilayer perceptron with PyTorch (two hidden layers of 64 units, Adam, early stopping on a validation split) on the standardized one-hot features, and compare its test accuracy with the gradient boosting. Then apply the same *random rotation* to the features of both: which model suffers?

Exercise 13 (A tabular foundation model: TabICL ⋆⋆)
Tabular foundation models (TabPFN, TabICL, Lecture 3) are transformers pretrained on millions of synthetic datasets: they predict the test labels *in context*, from the training set, without training nor tuning. TabICL v2 (2026) has open weights; TabPFN (2.5 and later) requires accepting a non-commercial license and a token, which is why we use TabICL here.

- Compute the gain of the Bayes decision on the probabilities of `TabICLClassifier`, and compare it with the CV gains of the core part (and with the test gain of the best model).

Exercise 14 (Synthetic tabular data ⋆⋆)
Synthetic data are generated by a model of the joint law of the features (and the label). A simple generator: a Gaussian mixture on the one-hot encoded and standardized features, for each class. A synthetic dataset is useful if a model trained on it performs well on *real* data (*train on synthetic, test on real*).

- Fit, for each class, a Gaussian mixture with 10 components (diagonal covariances) on the preprocessed training data, sample a synthetic training set of the same size and proportions, train a random forest on it, and compare its test accuracy with the one trained on the real data.

Exercise 15 (Fairness of the selected model ⋆⋆)
The dataset is IBM's *biased* version of the German credit data, and contains sensitive attributes (`Sex`, `ForeignWorker`, `Age`). `apm_credit.fairness_audit` computes, for each attribute, the accuracy, the approval rate and the true and false positive rates by group, as well as the demographic parity difference (difference of approval rates) and the equalized odds difference (Lecture 4, *Metrics and ethics*).

- Audit the Bayes decisions of the best model of the core part on the test set. Then retrain it without `Sex` and `ForeignWorker`: are the differences reduced? Why is *fairness through unawareness* not enough?

Exercise 16 (**AutoGluon (separate notebook)** $\star\star$)

AutoGluon trains many models (boosting libraries, random forests, neural networks, foundation models) and *stacks* them in several layers, with a time budget. As it pins many packages, it has its own short notebook, `APM_50441_EP_Lab_3_AutoGluon.ipynb`: open it on Colab (*Open in Colab* link), or run it on your own computer in a separate environment (see its *Working environment*). It covers AutoGluon in three minutes, what is inside (bagging and stacking), and the tabular foundation models TabICLv2 and TabDPT used through AutoGluon, compared with the models of this lab.

Exercise 17 (**A tiny prior-fitted network** $\star\star\star$)

A *prior-fitted network* (PFN, the idea behind TabPFN) is a transformer trained on many small *synthetic* datasets drawn from a prior: it receives a training set and test inputs, and learns to output the posterior predictive distribution of the test labels. Here, the prior generates 1D regression datasets $y = a\sin(\omega x + \phi) + \varepsilon$ with random a, ω, ϕ; after training, the network predicts *in context* on a new dataset, without any gradient step.

- Train a small transformer (PyTorch, 2 layers, the training points as tokens (x_i, y_i), the test points as tokens $(x_j, 0)$ attending only to the training tokens) on 3000 batches of 32 synthetic datasets of 20 training and 20 test points. Compare its predictions on a new dataset with a kernel ridge regression.

Use of AI. Artificial intelligence tools have been used to revise the content of this lab. The main contributor takes full responsibility for this content.

License. The texts of this lab are under the Creative Commons Attribution-ShareAlike 4.0 license (CC BY-SA 4.0), and its code under the MIT license; the datasets keep their own licenses. Main contributor: E. Le Pennec. Contributors: S. Boucheron, A. Dieuleveut, A.K. Fermin, S. Gadat, S. Gaiffas, B. Goujaud, A. Guilloux, Ch. Keribin, Th. Klein, E. Matzner, J. Read, M. Sangnier, E. Scornet.